

Week 3: Introduction to Machine Learning

Name: Nidhi Shankaranand Pattar

AI Intern — InternNova

Date: 20/08/2026

Task 1: Machine Learning Fundamentals

1. What is Machine Learning?

A subset of AI where systems learn patterns from data instead of being explicitly programmed with fixed rules, and use those patterns to make predictions on new, unseen data.

2. AI vs ML vs Deep Learning

Artificial Intelligence (AI) is the broad field of creating systems that can perform tasks that normally require human intelligence. Examples include reasoning, decision-making, speech recognition, and image understanding.

Machine Learning is a subset of AI. It allows computers to learn from data and improve their performance without being directly programmed for every task.

Deep Learning is a subset of Machine Learning that uses artificial neural networks with multiple layers. Deep learning is particularly useful for complex tasks involving images, speech, natural language, and large datasets.

The relationship can be represented as:

Artificial Intelligence → Machine Learning → Deep Learning

For example, a voice assistant is an AI application. Machine learning can be used to understand the user's speech, while deep learning can be used to recognize complex speech and language patterns.

3. What is a dataset?

A structured collection of data (usually rows and columns) used to train and test a machine learning model — e.g., a table of students with their study hours, attendance, and pass/fail result.

4. Features and Labels

Features = the input variables used to make a prediction (e.g., Study Hours, Attendance).

Label (also called target) = the output/answer you're trying to predict (e.g., Pass/Fail).

5. Training Data vs Testing Data

Training data = the portion of the dataset used to teach the model patterns.

Testing data = a separate, unseen portion used to check how well the model actually learned — evaluates real performance, not memorization.

6. Supervised Learning

The model learns from labeled data — meaning each example already has the correct answer, and the model learns the relationship between features and that answer. E.g., predicting Pass/Fail using past student data.

7. Unsupervised Learning

The model works with **unlabeled** data — no correct answers given. It finds hidden patterns or groupings on its own. E.g., grouping customers into segments based on purchase behavior, without being told the groups beforehand.

8. Classification vs Regression

Classification = predicting a category/class (e.g., Pass or Fail, Spam or Not Spam).

Regression = predicting a continuous number (e.g., predicting exact marks, house price).

9. Real-world applications

Spam email detection, product recommendations (Netflix/Amazon), fraud detection in banking, medical diagnosis prediction, house price estimation.

CONCLUSION

Machine Learning enables computers to learn from data and improve predictions over time without being explicitly reprogrammed for every scenario. Understanding concepts like features, labels, and the difference between supervised/unsupervised learning and classification/regression forms the foundation for building any ML model — skills directly applied in this week's mini project predicting student pass/fail outcomes.

Task 2: Dataset Preparation

Objective: To prepare a dataset for machine learning using Pandas by loading, inspecting, cleaning, and separating it into features and target.

Operations Performed: *(bullet list — loaded dataset, displayed first 5 rows, checked shape, checked data types, identified and handled missing values, separated into X (features) and y (target))*

```
[1]: import pandas as pd
```

```
[2]: df = pd.read_csv("student_performance.csv")
df
```

```
[3]:
```

	Student_Name	Marks	Attendance
0	Aarav	78.0	92
1	Diya	65.0	85
2	Rohan	92.0	96
3	Ananya	NaN	78
4	Kabir	55.0	80
5	Isha	88.0	94
6	Arjun	34.0	68
7	Meera	72.0	88
8	Rahul	45.0	75
9	Sneha	81.0	91
10	Vikram	29.0	60
11	Pooja	67.0	86
12	Rohan	92.0	96
13	Ananya	74.0	89

```
[3]: df.head()
```

```
[3]:
```

	Student_Name	Marks	Attendance
0	Aarav	78.0	92
1	Diya	65.0	85
2	Rohan	92.0	96
3	Ananya	NaN	78
4	Kabir	55.0	80

```
[4]: print("Rows and Columns:", df.shape)
Rows and Columns: (14, 3)
```

```
[5]: df.dtypes
```

```
[5]: Student_Name      str
Marks              float64
Attendance         int64
dtype: object
```

```
[6]: df.isnull().sum()
```

```
[6]: Student_Name      0
Marks              1
Attendance         0
dtype: int64
```

```
[7]: print("Duplicates before:", df.duplicated().sum())
df = df.drop_duplicates()
print("Duplicates after:", df.duplicated().sum())
Duplicates before: 1
Duplicates after: 0
```

```
[12]: print(df.columns.tolist())
['Student_Name', 'Marks', 'Attendance']
```

```
[15]: features = ['Marks', 'Attendance']
target = 'Result'
```

```
[20]: df = pd.read_csv("student_performance.csv")
df = df.drop_duplicates() # keep your existing duplicate removal

# NEW: fill missing Marks with the average
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())

print("Missing values after cleaning:")
print(df.isnull().sum()) # should now show 0 for Marks

Missing values after cleaning:
Student_Name      0
Marks              0
Attendance         0
dtype: int64
```

```
[16]: df['Result'] = df['Marks'].apply(lambda x: 'Pass' if x >= 40 else 'Fail')
```

```
features = ['Marks', 'Attendance']  
target = 'Result'
```

```
X = df[features]  
y = df[target]
```

```
print("Features (X):")  
print(X)
```

```
print("\nTarget (y):")  
print(y)
```

```
Features (X):  
   Marks  Attendance  
0    78.0         92  
1    65.0         85  
2    92.0         96  
3     NaN         78  
4    55.0         80  
5    88.0         94  
6    34.0         68  
7    72.0         88  
8    45.0         75  
9    81.0         91  
10   29.0         60  
11   67.0         86  
13   74.0         89
```

```
Target (y):  
0    Pass  
1    Pass  
2    Pass  
3    Fail  
4    Pass  
5    Pass  
6    Fail  
7    Pass  
8    Pass  
9    Pass  
10   Fail  
11   Pass  
13   Pass  
Name: Result, dtype: str
```

```
[16]: df['Result'] = df['Marks'].apply(lambda x: 'Pass' if x >= 40 else 'Fail')

features = ['Marks', 'Attendance']
target = 'Result'

X = df[features]
y = df[target]

print("Features (X):")
print(X)

print("\nTarget (y):")
print(y)
```

```
Features (X):
   Marks  Attendance
0   78.0         92
1   65.0         85
2   92.0         96
3    NaN         78
4   55.0         80
5   88.0         94
6   34.0         68
7   72.0         88
8   45.0         75
9   81.0         91
10  29.0         60
11  67.0         86
13  74.0         89
```

```
Target (y):
0    Pass
1    Pass
2    Pass
3    Fail
4    Pass
5    Pass
6    Fail
7    Pass
8    Pass
9    Pass
10   Fail
11   Pass
13   Pass
Name: Result, dtype: str
```

Explanation: Briefly state which columns you used as X and which as y, and why.

Task 3: Train-Test Split & Preprocessing (structure)

Objective: To split the dataset into training and testing sets using Scikit-learn and prepare it for model training.

Operations Performed: (*imported train_test_split, chose test_size, applied any preprocessing*)

```
[1]: import pandas as pd
from sklearn.model_selection import train_test_split
```

```
[2]: df = pd.read_csv("student_performance.csv")
```

```
print(df.head())
print(df.columns)
```

	Student_Name	Marks	Attendance
0	Aarav	78.0	92
1	Diya	65.0	85
2	Rohan	92.0	96
3	Ananya	NaN	78
4	Kabir	55.0	80

Index(['Student_Name', 'Marks', 'Attendance'], dtype='str')

```
[3]: df['Result'] = df['Marks'].apply(
    lambda x: 'Pass' if x >= 40 else 'Fail'
)
```

```
print(df.head())
```

	Student_Name	Marks	Attendance	Result
0	Aarav	78.0	92	Pass
1	Diya	65.0	85	Pass
2	Rohan	92.0	96	Pass
3	Ananya	NaN	78	Fail
4	Kabir	55.0	80	Pass

```
[5]: features = ['Marks', 'Attendance']
target = 'Result'
```

```
X = df[features]
y = df[target]
```

```
print("Features (X):")
print(X)
```

```
print("\nTarget (y):")
print(y)
```

Features (X):

	Marks	Attendance
0	78.0	92
1	65.0	85
2	92.0	96
3	NaN	78

```
[1]: import pandas as pd
from sklearn.model_selection import train_test_split
```

```
[2]: df = pd.read_csv("student_performance.csv")
```

```
print(df.head())
print(df.columns)
```

	Student_Name	Marks	Attendance
0	Aarav	78.0	92
1	Diya	65.0	85
2	Rohan	92.0	96
3	Ananya	NaN	78
4	Kabir	55.0	80

Index(['Student_Name', 'Marks', 'Attendance'], dtype='str')

```
[4]: df['Result'] = df['Marks'].apply(
    lambda x: 'Pass' if x >= 40 else 'Fail'
)
```

```
print(df.head())
```

	Student_Name	Marks	Attendance	Result
0	Aarav	78.0	92	Pass
1	Diya	65.0	85	Pass
2	Rohan	92.0	96	Pass
3	Ananya	NaN	78	Fail
4	Kabir	55.0	80	Pass

```
[5]: features = ['Marks', 'Attendance']
target = 'Result'
```

```
X = df[features]
y = df[target]
```

```
print("Features (X):")
print(X)
```

```
print("\nTarget (y):")
print(y)
```

Features (X):

	Marks	Attendance
0	78.0	92
1	65.0	85
2	92.0	96
3	NaN	78

```
[1]: import pandas as pd
from sklearn.model_selection import train_test_split

[2]: df = pd.read_csv("student_performance.csv")

print(df.head())
print(df.columns)
```

	Student_Name	Marks	Attendance
0	Aarav	78.0	92
1	Diya	65.0	85
2	Rohan	92.0	96
3	Ananya	NaN	78
4	Kabir	55.0	80

Index(['Student_Name', 'Marks', 'Attendance'], dtype='str')

```
[4]: df['Result'] = df['Marks'].apply(
    lambda x: 'Pass' if x >= 40 else 'Fail'
)

print(df.head())
```

	Student_Name	Marks	Attendance	Result
0	Aarav	78.0	92	Pass
1	Diya	65.0	85	Pass
2	Rohan	92.0	96	Pass
3	Ananya	NaN	78	Fail
4	Kabir	55.0	80	Pass

```
[5]: features = ['Marks', 'Attendance']
target = 'Result'

X = df[features]
y = df[target]

print("Features (X):")
print(X)

print("\nTarget (y):")
print(y)
```

Features (X):

	Marks	Attendance
0	78.0	92
1	65.0	85
2	92.0	96
3	NaN	78

```
print(X_train)
print(X_test)
print(y_train)
print(y_test)
```

X_train:

	Marks	Attendance
12	92.0	96
5	88.0	94
8	45.0	75
2	92.0	96
1	65.0	85
13	74.0	80
4	55.0	80
7	72.0	88
10	29.0	60
3	NaN	78
6	34.0	68

X_test:

	Marks	Attendance
9	81.0	91
11	67.0	86
0	78.0	92

y_train:

	Result
12	Pass
5	Pass
8	Pass
2	Pass
1	Pass
13	Pass
4	Pass
7	Pass
10	Fail
3	Fail
6	Fail

Name: Result, dtype: str

y_test:

	Result
9	Pass
11	Pass
0	Pass

Name: Result, dtype: str

The dataset is divided into training and testing data so that the machine learning model can learn patterns from the training data and then be evaluated on unseen testing data. In this project, 80% of the data is used for training and 20% for testing. Marks and Attendance are selected as features, while Result is the target.

Explanation (must include this explicitly): "Training Data → Model Learning → Testing Data → Prediction" — the model learns patterns from the training data, and its performance is then evaluated on the testing data, which it has never seen before, to check how well it generalizes to new situations.

Task 4: First ML Model (structure)

Objective: State whether you chose Classification (Logistic Regression/Decision Tree) or Regression (Linear Regression), and why.

```
[1]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load and clean - do this ONCE
df = pd.read_csv("student_performance.csv")
df = df.drop_duplicates()
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())

print("Missing values after cleaning:")
print(df.isnull().sum())

Missing values after cleaning:
Student_Name    0
Marks           0
Attendance      0
dtype: int64
```

```
[2]: df['Result'] = df['Marks'].apply(
    lambda x: 'Pass' if x >= 40 else 'Fail'
)

print(df.head())
```

	Student_Name	Marks	Attendance	Result
0	Aarav	78.0	92	Pass
1	Diya	65.0	85	Pass
2	Rohan	92.0	96	Pass
3	Ananya	65.0	78	Pass
4	Kabir	55.0	80	Pass

```
[3]: features = ['Marks', 'Attendance']
target = 'Result'

X = df[features]
y = df[target]

print("Features (X):")
print(X)

print("\nTarget (y):")
print(y)

Features (X):
   Marks  Attendance
0   78.0         92
1   65.0         85
```

```

Features (X):
  Marks  Attendance
0   70.0         92
1   65.0         85
2   92.0         96
3   65.0         78
4   55.0         80
5   88.0         94
6   34.0         86
7   72.0         88
8   45.0         75
9   81.0         81
10  29.0         68
11  87.0         86
13  74.8         89

Target (y):
0  Pass
1  Pass
2  Pass
3  Pass
4  Pass
5  Pass
6  Fail
7  Pass
8  Pass
9  Pass
10 Fail
11 Pass
13 Pass
Name: Result, dtype: str

```

```

[4]: X_train, X_test, y_train, y_test = train_test_split(
      X,
      y,
      test_size=0.2,
      random_state=42
    )

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

Training data: (10, 2)
Testing data: (3, 2)

[5]: model = DecisionTreeClassifier(random_state=42)

model.fit(X_train, y_train)

print("Model trained successfully!")

```

```

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

```

```

Training data: (10, 2)
Testing data: (3, 2)

```

```

[5]: model = DecisionTreeClassifier(random_state=42)

model.fit(X_train, y_train)

print("Model trained successfully!")

Model trained successfully!

```

```

[6]: y_pred = model.predict(X_test)

print("Actual Results:")
print(y_test.values)

print("\nPredicted Results:")
print(y_pred)

```

```

Actual Results:
<StringArray>
['Pass', 'Pass', 'Pass']
Length: 3, dtype: str

```

```

Predicted Results:
['Pass' 'Pass' 'Pass']

```

```

[7]: accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)
print("Model Accuracy (%):", accuracy * 100)

Model Accuracy: 1.0
Model Accuracy (%): 100.0

```

```
[ ]:
```

Explanation: How the model works, in your own words.

Task 5: Mini Project (structure)

```
[1]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load and clean - do this ONCE
df = pd.read_csv("student_performance.csv")
df = df.drop_duplicates()
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())

print("Missing values after cleaning:")
print(df.isnull().sum())
```

```
Missing values after cleaning:
Student_Name    0
Marks           0
Attendance      0
dtype: int64
```

```
[6]: df['Result'] = df['Marks'].apply(
    lambda x: 'Pass' if x >= 40 else 'Fail'
)

print(df)
```

	Student_Name	Marks	Attendance	Result
0	Aarav	78.0	92	Pass
1	Diya	65.0	85	Pass
2	Rohan	92.0	96	Pass
3	Ananya	65.0	78	Pass
4	Kabir	55.0	80	Pass
5	Isha	88.0	94	Pass
6	Arjun	34.0	68	Fail
7	Meera	72.0	88	Pass
8	Rahul	45.0	75	Pass
9	Sneha	81.0	91	Pass
10	Vikram	29.0	60	Fail
11	Pooja	67.0	86	Pass
13	Ananya	74.0	89	Pass

```
[7]: features = ['Marks', 'Attendance']
target = 'Result'

X = df[features]
y = df[target]

print("Features:")
print(X)
```

```
[7]: features = ['Marks', 'Attendance']
    target = 'Result'

    X = df[features]
    y = df[target]

    print("Features:")
    print(X)

    print("\nTarget:")
    print(y)

    Features:
       Marks  Attendance
0    78.0         92
1    65.0         85
2    92.0         96
3    65.0         78
4    55.0         88
5    88.0         94
6    34.0         68
7    72.0         88
8    45.0         75
9    81.0         91
10   29.0         68
11   67.0         86
13   74.0         89

    Target:
0    Pass
1    Pass
2    Pass
3    Pass
4    Pass
5    Pass
6    Fail
7    Pass
8    Pass
9    Pass
10   Fail
11   Pass
13   Pass
Name: Result, dtype: str

[8]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 10
Testing samples: 3
```

```
[10]: model = DecisionTreeClassifier(random_state=42)

model.fit(X_train, y_train)

print("Decision Tree model trained successfully!")

Decision Tree model trained successfully!
```

```
[11]: y_pred = model.predict(X_test)

print("Actual Results:")
print(y_test.values)

print("\nPredicted Results:")
print(y_pred)
```

```
Actual Results:
<StringArray>
['Pass', 'Pass', 'Pass']
Length: 3, dtype: str
```

```
Predicted Results:
['Pass' 'Pass' 'Pass']
```

```
[12]: accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)
print("Model Accuracy:", accuracy * 100, "%")
```

```
Model Accuracy: 1.0
Model Accuracy: 100.0 %
```

```
[13]: new_students = pd.DataFrame({
    'Marks': [75, 35, 55],
    'Attendance': [90, 60, 80]
})

predictions = model.predict(new_students)

new_students['Prediction'] = predictions

print("New Student Predictions:")
print(new_students)
```

```
'Marks': [75, 35, 55],
'Attendance': [90, 60, 80]
})
```

```
predictions = model.predict(new_students)
```

```
new_students['Prediction'] = predictions
```

```
print("New Student Predictions:")
print(new_students)
```

```
New Student Predictions:
  Marks  Attendance Prediction
0    75         90      Pass
1    35         60      Fail
2    55         80      Pass
```

```
[14]: for i in range(len(new_students)):
      print(
          f"Student {i+1}: "
          f"Marks = {new_students.iloc[i]['Marks']}, "
          f"Attendance = {new_students.iloc[i]['Attendance']}%, "
          f"Prediction = {new_students.iloc[i]['Prediction']}"
      )
```

```
Student 1: Marks = 75, Attendance = 90%, Prediction = Pass
Student 2: Marks = 35, Attendance = 60%, Prediction = Fail
Student 3: Marks = 55, Attendance = 80%, Prediction = Pass
```

Student Performance Prediction — Report 1

This mini project focuses on predicting whether a student is likely to Pass or Fail using Machine Learning. The dataset was loaded using Pandas and contained student names, marks, and attendance. The data was checked for missing values and duplicate records before preparing it for the Machine Learning model.

The features selected for prediction were Marks and Attendance. The target variable was Result, where students with marks of 40 or above were considered Pass and students with marks below 40 were considered Fail. The dataset was divided into training and testing data using an 80:20 ratio.

A Decision Tree Classifier from Scikit-learn was used as the Machine Learning model. The model learned patterns from the training data and was then used to predict the results of the testing data. Model accuracy was calculated using `accuracy_score`. The model was also tested with three new student inputs to demonstrate real-world prediction.

The model achieved a high accuracy on the test data. However, this result should be interpreted carefully because the Result column was created directly from Marks using a fixed pass-mark rule. Therefore, the model is mainly demonstrating the Machine Learning process rather than providing a highly realistic prediction.

In the future, the project could be improved by using a larger real-world dataset and additional features such as study hours, previous marks, assignment scores, project scores, and participation. Different classification algorithms could also be compared to determine which model performs best.

Conclusion

This week focused on the foundations of Machine Learning, moving from core concepts to actually building a working model. Starting with understanding features, labels, and the difference between supervised and unsupervised learning, the assignment progressed through preparing a real dataset, splitting it into training and testing sets, and training a Decision Tree Classifier to predict student outcomes. The mini project tied these concepts together by building a complete pipeline — from raw data to a trained model making real predictions on new student inputs. This week highlighted that the biggest part of Machine Learning isn't just training a model, but properly preparing and understanding the data beforehand, since a model is only as reliable as the data it learns from.

